Final Project Design Phase

Computer Science - Week 11

Oct 3, 2026 - Version 0.0.4

For complete instructions and project ideas, refer to the assignment in Canvas. Specifically, review the final project requirements before you continue below. If you have not reviewed the project requirements in Canvas (you might have to go to the modules view and check the current or past week's content), you will not be able to properly complete the parts below. In addition, it may be helpful to review the information on this Drafter documentation page.

0) Project Planning and Preparation

Successful programmers spend time planning before they begin coding. *After* reviewing the project requirements, answer the following questions to help guide your design and development process.

A. What concepts from class or previous assignments might help you complete this project? Which programming concepts (loops, conditionals, lists, classes, etc.) do you expect to use? What programming skills do you want to improve while completing this project?

I will need to use loops to search through a list, I will need conditionals to check if things are the same. I will use dataclasses as my datapoints, each user input will go into an attribute.

B. Which parts of the project do you expect to be the easiest for you? The hardest? Why?

I expect the loops and searching to be the easiest and the drafter heavy portions to be the hardest because I still struggle with understanding drafter concepts.

C. Even though you have not yet designed the final project, estimate which tasks will likely require the most time and effort? Which features or components should be implemented first?

I think creating the dataset of options for birds will take the longest because not only will I need to create the data I will need to do research to do so. I will likely make a

separate file to use throughout the program.

D. What distractions might interfere with your work and how will you minimize them? What resources will you need available while working? What questions, if any, do you still have about the assignment requirements?

Other distractions will be my work for other classes, I will minimize this by setting aside specific time to this project.

1) Site Diagram

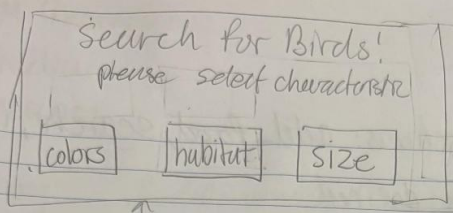
Create a mock-up of your planned website. We suggest you make the mockup on paper, although you can also use software like PowerPoint or Slides to make a mockup. There is other design software that could also be used, but we strongly encourage you to keep it simple. Do not spend more than 15-30 minutes detailing the mock-up. Your goal should be to make a very quick and dirty diagram, to show that you have some kind of idea.

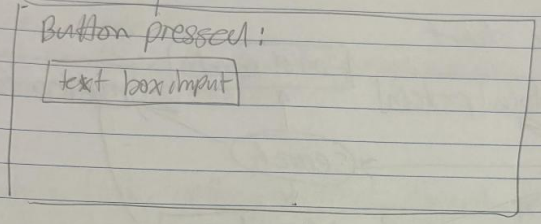
We recommend drawing each page as a rectangle, and laying out the text, buttons, and other components inside. Then, use arrows to indicate links between pages. Label each page the way you will name each function, and then put labels on the arrows to indicate what the button links. Also write down any parameters that should come into the pages.

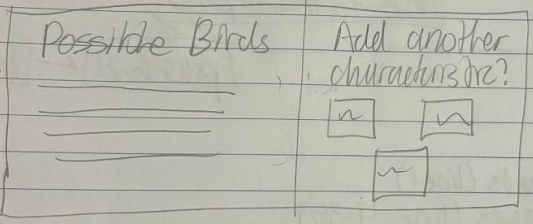
In addition to the pages and routes, write down what fields your `State` will have, both their names and types. Think clearly about what data will need to be persistent across the entire website.

Add photo(s) or screenshot(s) of your website mock-up and labeled diagram on the next page.

1st page







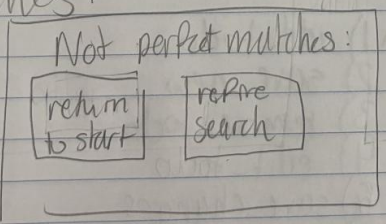
list of possible matches

buttons not pressed

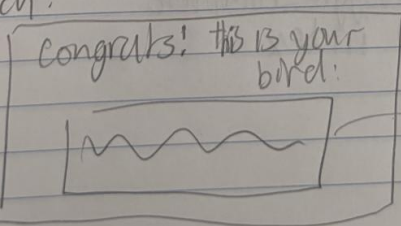
→ repeated page w/ new characteristics

rank choices by matching

if no matches:



perfect match:



name of bird w/ characteristics

2) Implementing Requirements

This table should serve as a draft for how you will implement the requirements into your final project. You may change your ideas later on, but your final site must fulfill all the requirements.

Requirement	**Implementation**
Purpose/Functionality A 1-2 sentence description of the purpose or functionality of your site	Create a search engine for finding birds, use user input in specific categories to search a dataset.
State Dataclass Attributes Minimum of 4 attributes, one of which must be a list/dict of instances of your second dataclass. Briefly describe each attribute, its type, and how it will be meaningfully modified throughout the site.	State: color: str habitat: str size: str Each attribute is empty to start, changed by user input, each has specific options Habitat: field, forest, bush, sky Size: small, med, large Color: white, black, brown, tan, blue, …
Second Dataclass Your second dataclass, including its attributes, their types, and how they will be meaningfully modified (if they are modified)	Bird: main_color: list[str] habitat: list[str] size: str These represent the list of birds from a dataset- each turned into a dataclass to be able to search through depending on user changes to state

Routes (minimum 7) The names of your route functions	Index after_colors- user selects from colors After_habitat- selects from habitats After_size- selects from sizes return_to_start- removes all changes to state and returns start page refine_search- back to choosing buttons but saves prior changes to state show_results- shows results from selections
Pages (minimum 5) A brief description of the pages in your site	Og page- options to search Page after color search- birds that match color Page after habitat search- birds that match habitat Page after size search- birds that match size No matches- based on three characteristics, no matches Perfect match- specific bird that matches all 3
User Input (minimum 3) Description of user input components (TextBox, TextArea, CheckBox, or SelectBox) and their purposes For more information on these input types, see Drafter Documentation.	Selectbox- users select different options for colors and other attributes- once for each (3 total) Other buttons to correspond to user choices (refine, restart, etc) Each attribute will also have an “unsure” button that leaves the category blank
If Statements (minimum 3) Brief description of what will be evaluated in your if statements and how they will meaningfully change the logic of the site	If color, habitat, and size all match for a specific bird then it returns the perfect match page If the selections do not match any then return the no matches page If some match return list of possible

Loop (minimum 1) This loop must iterate over the list/dict of dataclass instances in your State attribute. Description of this loop and why it is meaningful to the site	Check over list of Bird dataclasses and lists within dataclass- this is the ultimately how the website works- it checks the user input against already known data

LLM Conversation

If you used an LLM in any part of this work, copy and paste every part of that conversation below. Otherwise, you may be given a zero for the assignment. Obviously, these words will not all be yours, and this is the exception to the requirement at the top of this assignment that all responses be in your own words. If you did not use an LLM to assist you in this assignment, simply say so below.

i am making a project that is like a search engine for birds, I have these route functions so far, but I need help coming up with another and refining these so they are actually possible: Index

after_colors

after_habitat

after_size

return_to_start- removes all changes to state and returns start page

refine_search- back to choosing buttons but saves prior changes to state

I think your bird search engine is very achievable in Drafter, especially if you keep it as

a guided filtering system rather than trying to build a Google-style search engine.

Your current routes make sense, but I'd refine their responsibilities so each route has a clear purpose and is something you can realistically implement with basic Python.

1. Refining your existing routes

Route	Purpose	Feasibility
`index`	Displays the welcome page and starts the bird search.	Easy
`after_colors`	Displays color choices for the user to select.	Easy
`after_habitat`	Displays habitat choices (forest, ocean, grassland, etc.).	Easy
`after_size`	Displays bird size choices (small, medium, large).	Easy

`refine_search`	Returns the user to the filtering options while preserving their previous selections.	Easy
`return_to_start`	Resets all search selections and returns to the welcome page.	Easy

One important suggestion: I would make `refine_search` preserve the user's previous selections so they can change just one answer without starting over.

For example, if someone selected:

- Color: Blue
- Habitat: Ocean
- Size: Small

They could return to the filters and change their habitat to Forest without losing their color and size selections.

2. My recommendation for your additional route

```
show_results
```

Recommended



Purpose: Takes the user's selected filters and displays birds that match those characteristics.

For example, if the user selects:

- Blue
- Forest
- Small

The results page displays birds in your dataset that match those characteristics.

The user could see:

- Bird name
- Picture (if available)
- Color
- Habitat
- Size

This is probably the most important route because it completes the actual search engine functionality.

3. How I would organize the entire application

Here is the proposed navigation flow:

index: Welcome Pageafter_colors: Choose Colorsafter_habitat: Choose Habitatafter_size: Choose Sizeshow_results: Display MatchingBirdsrefine_search: Edit Filtersreturn_to_start: Reset Search

You could also let users navigate between the color, habitat, and size pages without having to restart their search.

4. One important technical consideration: State

Since Drafter runs in the browser using Python and doesn't require a backend or database, you can keep your bird information in a Python list of dictionaries.

For example:

```
birds = [

    {

        "name": "Blue Jay",
```

```
        "color": "Blue",

        "habitat": "Forest",

        "size": "Medium"

    },

    {

        "name": "Eastern Bluebird",

        "color": "Blue",

        "habitat": "Grassland",

        "size": "Small"

    }

]
```

You can also store the user's selections in Drafter's application state:

```
state.selected_color


state.selected_habitat


state.selected_size
```

Then your `show_results` route can loop through the bird list and compare each bird's characteristics against the user's selections.

This is much more realistic than trying to connect to an external bird database or build a complicated search algorithm.

5. Final recommended route list

I would submit this as your planned route structure:

1. `index` – Welcome page and starting point.

2. `after_colors` – Allows the user to select bird colors.
3. `after_habitat` – Allows the user to select bird habitats.
4. `after_size` – Allows the user to select bird sizes.
5. `show_results` – Displays birds matching the selected characteristics.
6. `refine_search` – Returns to the filtering process while preserving previous selections.
7. `return_to_start` – Clears all selections and returns to the welcome page.

One design decision I'd like you to consider: Do you want users to select exactly one color, habitat, and size, or should they be able to select multiple colors and habitats?

For example, allowing someone to select both blue and yellow would make the search more flexible, but it slightly changes how you would write the filtering logic.

what would the pages look like for these routs

I would design your bird search engine as a simple, colorful bird-identification website. Each page should have one clear job, consistent styling, and large buttons so users can easily narrow down what bird they saw.

Here is how I would lay out each of your seven routes.

1. `index` — Welcome page

Starting page



Purpose: Introduce the website and let users begin identifying a bird.

Suggested layout:

- Website title: Bird Finder
- Subtitle: "Not sure what bird you saw? Let's find out!"
- Large bird illustration or photograph.
- Button: Start Searching
- Optional short instructions explaining that users will answer three questions.

2. `after_colors` — Color selection

Search step 1



Purpose: Ask the user what color the bird was.

Suggested layout:

What colors did the bird have?

Display selectable color buttons:

Blue

Red

Yellow

Green

Orange

Black

White

Brown

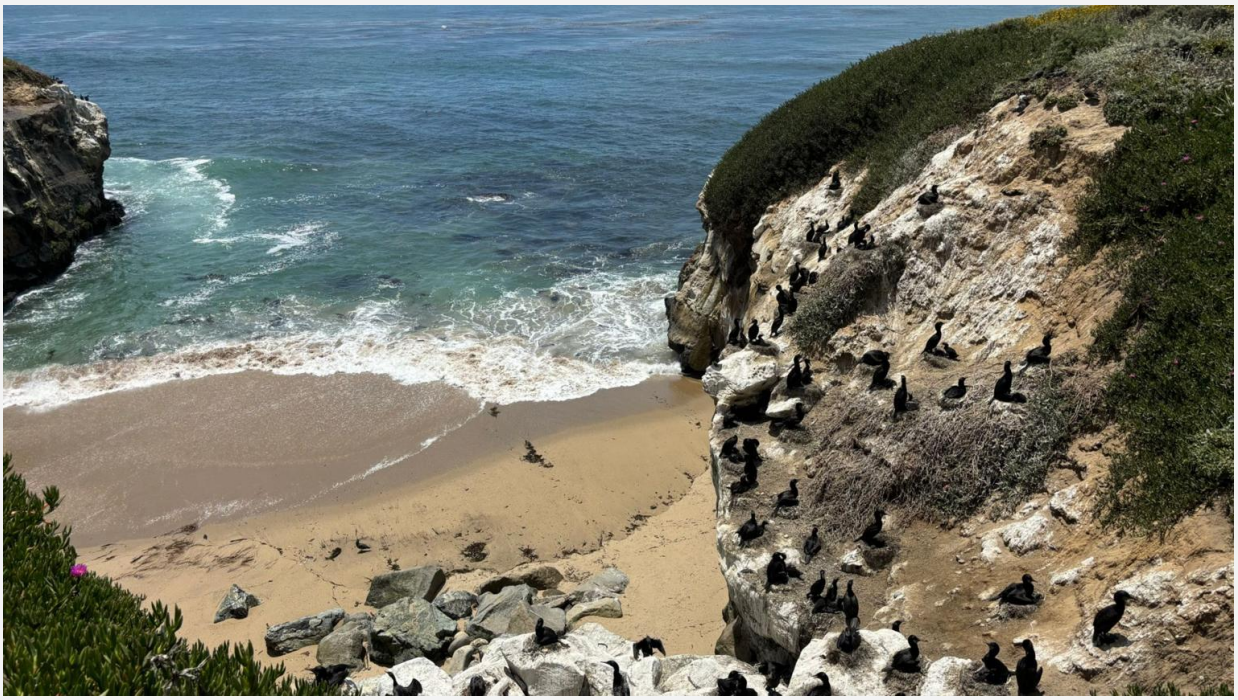
Gray

- Allow multiple colors to be selected.
- Button: Next
- Button: Back

3. `after_habitat` — Habitat selection

Search step 2







Purpose: Ask where the user saw the bird.

Suggested layout:

Where did you see the bird?

Display large selectable cards:



Forest



Grassland



Coastal

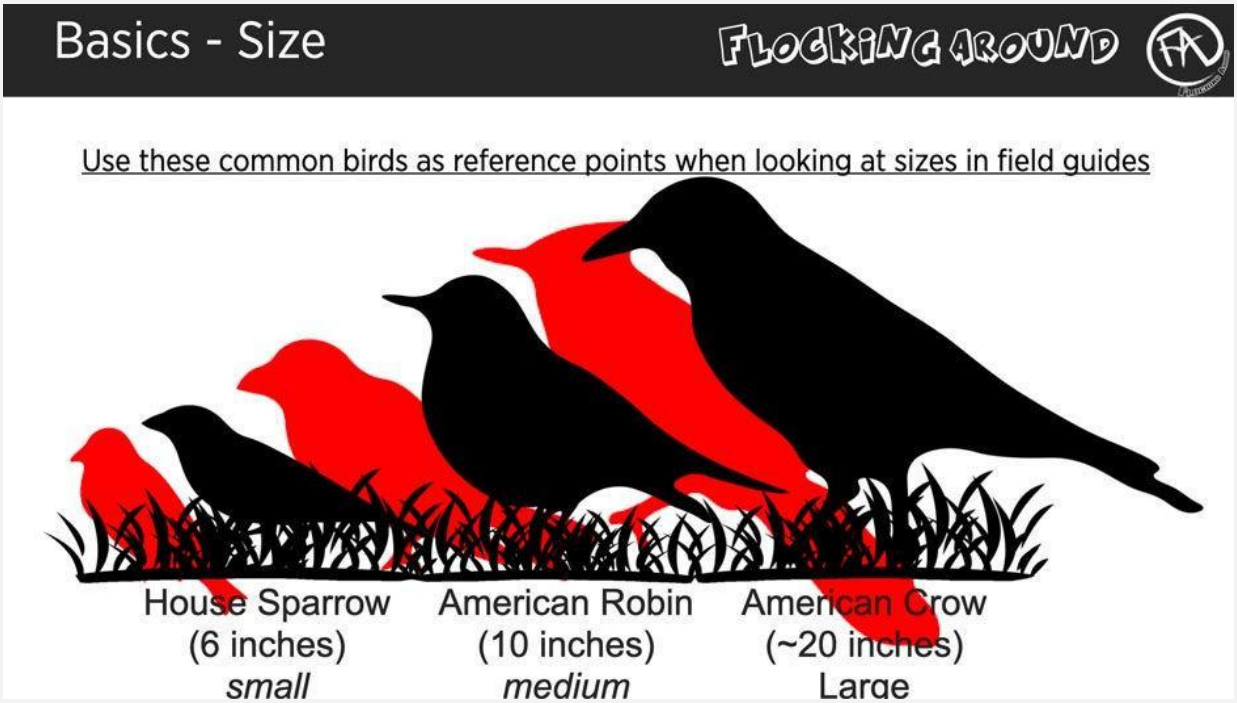


Wetland

- Allow multiple selections.
- Button: Next
- Button: Back

4. `after_size` — Size selection

Search step 3



Purpose: Ask how large the bird was.

Suggested layout:

How big was the bird?



Small

About the size of a sparrow



Medium

About the size of a robin



Large

About the size of a hawk or larger

- Allow one selection.
- Button: Find My Bird!
- Button: Back

5. `show_results` — Search results

Main results page

Purpose: Display birds matching the user's selections.

Suggested layout:

We found these birds for you!

Blue Jay

Match



- Color: Blue, white, black
- Habitat: Forest, woodland
- Size: Medium
- A crested bird with bright blue feathers and a distinctive call.

Eastern Bluebird



- Color: Blue, orange
- Habitat: Grassland, open woodland
- Size: Small
- A small songbird with a bright blue back and rusty-orange chest.

At the bottom:

- Refine Search button
- Start Over button

Important: These are example results. Your actual page should display only birds that match the selected filters.

6. `refine_search` — Edit your search

Purpose: Allow users to change their selections without losing everything.

Suggested layout:

Let's narrow down your search!

Show their current selections:

Your current filters

COLORS

BlueYellow

HABITAT

Forest

SIZE

Small

Choose a filter below to change your answer.

Buttons:

- Change Colors
- Change Habitat
- Change Size
- Update Results

The important part is that the user's previous choices remain saved in state.

7. `return_to_start` — Reset search

Purpose: Clear the user's selections and return to the welcome page.

This doesn't necessarily need to be a separate visually designed page!

I would actually make `return_to_start` a route function that:

1. Resets the selected colors.
2. Resets the selected habitat.
3. Resets the selected size.
4. Navigates back to `index`.

That way, users don't need to see an unnecessary extra page.

Overall design consistency

I recommend using the same visual style across all pages.



Suggested color palette

Forest green

Sage

Cream

Warm tan

Design principles

- Green and cream backgrounds.
- Rounded buttons and selection cards.
- Large bird photographs.
- Consistent navigation buttons.
- Clear instructions on every page.

My biggest recommendation for your CS1 project

Don't make seven completely different designs. Make three reusable page layouts:

1. Welcome layout: Used by `index`.
2. Question layout: Used by `after_colors`, `after_habitat`, and `after_size`.
3. Results layout: Used by `show_results` and potentially `refine_search`.

This will make your project much easier to code because you can reuse components and functions instead of recreating everything.

You can make a really nice-looking application without complicated programming. The most important programming challenge should be getting the filtering logic and state management to work correctly.

what would user input look like for this

For your bird search engine, I would make user input primarily consist of clickable buttons, image selections, and a few optional text fields, rather than having users type everything manually.

The goal is to make identifying a bird feel like answering a short quiz.

1. Color input (`after_colors`)

Question: What colors were on the bird?

Use selectable color buttons. Users should be able to select multiple colors.

What colors did you see?

Select all that apply.

Blue

Red

Yellow

Green

Orange

Black

White

Brown

Gray

Example selections: Blue, White

Python input: A list of selected colors.

2. Habitat input (`after_habitat`)

Question: Where did you see the bird?

Use image cards with labels. Users can select multiple habitats.

Where did you see it?

Select all that apply.

Forest

Selected

Grassland

Wetland

Coastal

Desert

Urban / Backyard

Python input: A list of selected habitats.

3. Size input (`after_size`)

Question: How big was the bird?

I recommend using radio buttons rather than checkboxes because most users will only need one size.

How big was it?

Small

Sparrow or hummingbird sized

Medium

Robin or blue jay sized

Large

Hawk or eagle sized

I'm not sure

Skip size as a search filter

Python input: One string representing the selected size.

4. An optional fourth input: Location

This is something I think could make your project more interesting.

Question: Where did you see the bird?

You could let users choose:

- My backyard
- A park
- Near water
- Somewhere else

Or provide an optional text box where they enter a state.

However, I would make this optional. Building a comprehensive geographic bird database is more complicated than your other filters.

5. What happens when the user submits?

After answering the questions, the user clicks Find My Bird!

For example, their input might look like this:

Your bird search

SELECTED COLORS

BlueWhite

SELECTED HABITATS

Forest

SELECTED SIZE

Medium

Your program then compares these answers against your bird list and displays matching birds.

6. How the input translates into Python

Your application could store the selections in Drafter state like this:

```
state.selected_colors = []


state.selected_habitats = []


state.selected_size = ""




state.birds = [
```

```
    {

        "name": "Blue Jay",

        "colors": ["Blue", "White", "Black"],

        "habitats": ["Forest", "Urban / Backyard"],

        "size": "Medium"

    }

]
```

The important distinction is:

User action	Python data type

Select multiple colors	List of strings
Select multiple habitats	List of strings
Select one size	String
Submit search	Button action

My recommendation

I would stick to these three required inputs:

1. Color
2. Habitat
3. Size

And include an "I'm not sure" option for each category.

This is important because someone might remember seeing a blue bird but have no idea how big it was.

Your filtering logic should simply ignore categories marked as unknown or left blank. That way, users can still find birds even when they don't remember every detail.

This is a realistic scope for a three-week CS1 project, and it gives you enough variety in input types to demonstrate lists, strings, conditionals, loops, and state management.

For a bird search app, here’s an option for deploying your web project.

Final Submission

When you are finished, download this file as a Word Document (docx) and upload the file to the appropriate assignment on Canvas.